

Practical No 04

Name : Pranjal somatkar Roll no. B 48

Aim: To perform numerical computations efficiently using the NumPy library in Python.

Theory

Q1. Explain the Role of NumPy in the Python Scientific Computing Ecosystem

NumPy (Numerical Python) is one of the most important libraries in Python for scientific computing. It provides powerful tools to handle numerical data efficiently, especially when working with large datasets and complex mathematical operations.

NumPy forms the **foundation of the Python scientific ecosystem**, enabling faster computation and supporting many advanced libraries used in data science, machine learning, and engineering.

Key Roles of NumPy

1. Efficient Multi-dimensional Array Handling

NumPy introduces the **ndarray (N-dimensional array)**, which allows storing large amounts of numerical data in a structured and efficient way.

Unlike Python lists, NumPy arrays:

- Use less memory
- Are faster
- Support advanced operations

2. Fast Numerical Computation

NumPy performs operations using **vectorization**, meaning calculations are applied to entire arrays without using loops.

This improves:

- Speed
- Performance
- Code simplicity

3. Mathematical and Statistical Operations

NumPy provides built-in functions for:

- Arithmetic operations
- Trigonometry
- Linear algebra
- Statistics

4. Backbone for Other Libraries

NumPy acts as a base for many popular Python libraries, making it essential in the ecosystem.

Table: Core Features of NumPy

Feature	Description	Example
ndarray	Multi-dimensional array	<code>np.array([1, 2, 3])</code>
Vectorization	Loop-free operations	<code>arr * 2</code>
Mathematical Functions	Built-in functions	<code>np.sqrt(arr)</code>
Memory Efficiency	Compact storage	Faster than lists
Broadcasting	Operations on different shapes	<code>arr + 5</code>

Example: NumPy vs Python List

```
import numpy as np

# Python list
lst = [1, 2, 3, 4]
result = [x * 2 for x in lst]

# NumPy array
arr = np.array([1, 2, 3, 4])
result_np = arr * 2

print(result_np)
```

Role in Scientific Ecosystem

NumPy is widely used in various fields:

Field	Role of NumPy
Data Science	Data analysis and preprocessing
Machine Learning	Input data handling
Engineering	Mathematical modeling
Research	Scientific simulations

Integration with Other Libraries

NumPy works closely with many libraries:

Library	How NumPy Helps
Pandas	Uses NumPy arrays for DataFrames
Matplotlib	Uses arrays for plotting graphs
Scikit-learn	Uses arrays for ML algorithms

Advantages of NumPy

- High performance
- Easy to use
- Supports large datasets
- Reduces code complexity
- Widely supported

Q2. Why is NumPy considered faster than pure Python computations?

NumPy is considered faster than pure Python computations because it is designed to perform numerical operations efficiently using optimized techniques and low-level implementations. While Python uses slower, high-level loops, NumPy performs operations in a faster and more optimized way.

Main Reasons for High Performance

1. Vectorization

NumPy uses **vectorized operations**, meaning it performs operations on entire arrays at once instead of using loops.

This reduces:

- Execution time
- Code complexity

2. Implementation in C Language

NumPy is built using **C and C++**, which are much faster than Python.

Advantages:

- Faster execution
- Efficient memory usage

3. Contiguous Memory Allocation

NumPy arrays store data in **continuous memory locations**, unlike Python lists.

Benefits:

- Faster access
- Better cache performance

4. Reduced Overhead

Python lists store different data types and require type checking, while NumPy arrays:

- Store same data type
- Avoid repeated type checking

5. Built-in Optimized Functions

NumPy provides highly optimized mathematical functions that run faster than manual implementations.

Table: NumPy vs Python

Feature	Python Lists	NumPy Arrays
Speed	Slow (uses loops)	Fast (vectorized)
Memory Usage	High	Low
Data Type	Mixed	Same type
Implementation	Pure Python	C-based
Performance	Low	High

Example: Performance Comparison

```
import numpy as np
import time

# Python list
lst = list(range(1000000))
start = time.time()
result = [x * 2 for x in lst]
print("Python time:", time.time() - start)

# NumPy array
arr = np.array(lst)
start = time.time()
result = arr * 2
print("NumPy time:", time.time() - start)
```

Output shows NumPy is significantly faster.

Real-Life Explanation

Imagine calculating marks for 1 million students:

- Python: Processes one by one (slow)
- NumPy: Processes all at once (fast)

Advantages of NumPy Speed

- Handles large datasets efficiently
- Reduces execution time
- Improves performance in data science tasks
- Ideal for real-time computations

Q3. What types of mathematical operations can be performed using NumPy?

NumPy provides a wide variety of mathematical operations that can be performed efficiently on arrays. These operations are essential for scientific computing, data analysis, and machine learning.

Types of Mathematical Operations in NumPy

NumPy supports different categories of operations:

1. Arithmetic Operations

These are basic mathematical operations performed element-wise on arrays.

Examples:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b) # Addition
print(a - b) # Subtraction
print(a * b) # Multiplication
print(a / b) # Division
```

2. Statistical Operations

Used to analyze data and find important values.

Common Functions:

- Mean
- Median
- Standard Deviation
- Variance

Example:

```
arr = np.array([10, 20, 30, 40])

print(np.mean(arr))
print(np.std(arr))
print(np.var(arr))
```

3. Trigonometric Operations

Used in mathematical and engineering calculations.

Functions:

- `sin()`, `cos()`, `tan()`

Example:

```
arr = np.array([0, np.pi/2, np.pi])

print(np.sin(arr))
print(np.cos(arr))
```

4. Logarithmic and Exponential Operations

Used in advanced computations.

Functions:

- `log()`, `log10()`, `exp()`

Example:

```
arr = np.array([1, 2, 3])

print(np.log(arr))
print(np.exp(arr))
```

5. Linear Algebra Operations

Used for matrix-related calculations.

Functions:

- Matrix multiplication
- Determinant

- Inverse

Example:

```
A = np.array([[1, 2], [3, 4]])

print(np.dot(A, A))
print(np.linalg.det(A))
```

6. Rounding Operations

Used to control decimal values.

Functions:

- `round()`, `floor()`, `ceil()`

Example:

```
arr = np.array([1.2, 2.7, 3.5])

print(np.round(arr))
print(np.floor(arr))
print(np.ceil(arr))
```

Summary Table of Operations

Category	Functions	Purpose
Arithmetic	+, -, *, /	Basic calculations
Statistical	mean, std, var	Data analysis
Trigonometric	sin, cos, tan	Angle calculations
Logarithmic	log, exp	Advanced math
Linear Algebra	dot, det	Matrix operations
Rounding	round, ceil	Approximation

Key Features

- Operations are **element-wise**
- Supports **vectorization**
- Works efficiently on large datasets
- Reduces need for loops

Q4. Explain Indexing and Slicing in NumPy Arrays

Indexing and slicing are powerful techniques in NumPy used to access and manipulate specific elements or subsets of an array. These operations help in efficiently handling data without modifying the original structure unnecessarily.

1. Indexing in NumPy

Indexing is used to access a **single element** from an array using its position.

Types of Indexing:

a. 1D Array Indexing

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr[0]) # First element
print(arr[2]) # Third element
```

b. 2D Array Indexing

```
arr2 = np.array([[1, 2], [3, 4]])

print(arr2[0, 1]) # Row 0, Column 1 → 2
```

ic. Negat ve Indexing

Access elements from the end.

```
print(arr[-1]) # Last element
```

2. Slicing in NumPy

Slicing is used to access **multiple elements (a subset)** of an array.

Syntax:

```
array[start : end : step]
```

Examples:

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4]) # Elements from index 1 to 3
print(arr[:3])  # First 3 elements
print(arr[::2]) # Step of 2
```

2D Array Slicing:

```
arr2 = np.array([[1,2,3],
                 [4,5,6]])

print(arr2[:, 1:3]) # All rows, columns 1 to 2
```

3. Boolean Indexing

Used to filter elements based on conditions.

```
arr = np.array([10, 20, 30, 40])

print(arr[arr > 20]) # Elements greater than 20
```

4. Fancy Indexing

Uses a list of indices to access elements.

```
arr = np.array([10, 20, 30, 40])

print(arr[[0, 2]]) # Access index 0 and 2
```

Table: Indexing vs Slicing

Feature	Indexing	Slicing
Purpose	Access single element	Access multiple elements
Syntax	<code>arr[i]</code>	<code>arr[start:end]</code>
Output	Scalar value	Array
Flexibility	Less	More

◆ Advantages of Indexing and Slicing

- Easy data access
- Efficient data manipulation
- Saves time and memory
- Works with multi-dimensional arrays

◆ Real-Life Example

If an array represents marks of students:

- Indexing → Get marks of one student
 - Slicing → Get marks of a group of students

Q5. How does NumPy integrate with libraries like Pandas, Matplotlib, and Scikit-learn?

NumPy integrates seamlessly with many popular Python libraries and acts as a **foundation for data science, visualization, and machine learning**. Libraries like Pandas, Matplotlib, and Scikit-learn rely heavily on NumPy arrays for efficient data handling and computation.

1. Integration with Pandas

Pandas is a data analysis library that uses NumPy arrays internally to store and manipulate data.

✓ Role of NumPy in Pandas:

- Provides **data storage (ndarray)**
- Enables **fast operations on DataFrames**
- Supports numerical computations

Example:

```
import pandas as pd
import numpy as np

data = np.array([[1, 2], [3, 4]])
df = pd.DataFrame(data, columns=['A', 'B'])

print(df)
```

Table: NumPy + Pandas

Feature	Role of NumPy
Data Storage	Uses ndarray
Speed	Faster processing
Operations	Numerical computations

2. Integration with Matplotlib

Matplotlib is used for data visualization and uses NumPy arrays as input for plotting graphs.

Role of NumPy in Matplotlib:

- Supplies **data for plotting**
- Handles **numerical calculations for graphs**
- Works with X and Y values

Example:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1, 2, 3, 4])
y = np.array([10, 20, 30, 40])

plt.plot(x, y)
plt.show()
```

Table: NumPy + Matplotlib

Feature	Role of NumPy
Input Data	Provides arrays
Graph Values	X and Y coordinates
Efficiency	Faster plotting

3. Integration with Scikit-learn

Scikit-learn is a machine learning library that uses NumPy arrays for model training and predictions.

Role of NumPy in Scikit-learn:

- Provides **input data format**
- Enables **fast mathematical computations**
- Supports **model training and testing**

Example:

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[1], [2], [3]])
y = np.array([2, 4, 6])

model = LinearRegression()
model.fit(X, y)

print(model.predict([[4]]))
```

Table: NumPy + Scikit-learn

Feature	Role of NumPy
Input Data	Arrays used
Computation	Fast processing
ML Models	Training support

Combined Integration Table

Library	Purpose	Role of NumPy
Pandas	Data Analysis	Data storage & operations
Matplotlib	Visualization	Graph plotting data
Scikit-learn	Machine Learning	Model computation

Advantages of Integration

- Smooth data flow between libraries
- High performance
- Reduced complexity
- Efficient handling of large datasets

Weak Students (Basic)

1. Create a NumPy array from a list

```
import numpy as np

# Creating array from list
lst = [1, 2, 3, 4, 5]
arr = np.array(lst)

print("NumPy Array:", arr)

NumPy Array: [1 2 3 4 5]
```

2. Perform basic arithmetic operations on arrays

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

```
Addition: [5 7 9]
Subtraction: [-3 -3 -3]
Multiplication: [ 4 10 18]
Division: [0.25 0.4 0.5 ]
```

3. Find sum and mean of array elements

```
import numpy as np

arr = np.array([10, 20, 30, 40])

print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))
```

```
Sum: 100
Mean: 25.0
```

4. Reshape a one dimensional array

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

reshaped = arr.reshape(2, 3)
print("Reshaped Array:\n", reshaped)
```

```
Reshaped Array:
[[1 2 3]
 [4 5 6]]
```

5. Access specific elements using indexing

```
import numpy as np

arr = np.array([10, 20, 30, 40])

print("First element:", arr[0])
print("Last element:", arr[-1])
```

```
First element: 10
Last element: 40
```

✓ Normal Students (Moderate)

6. Perform matrix addition and multiplication

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("Addition:\n", A + B)
print("Multiplication:\n", np.dot(A, B))
```

```
Addition:
[[ 6  8]
 [10 12]]
Multiplication:
[[19 22]
 [43 50]]
```

7. Generate random numbers using NumPy

```
import numpy as np

print("Random numbers:", np.random.rand(5))
print("Random integers:", np.random.randint(1, 10, size=5))
```

```
Random numbers: [0.75959529 0.57333531 0.59981299 0.53581843 0.05971728]
Random integers: [9 9 5 6 7]
```

8. Compute standard deviation and variance

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print("Standard Deviation:", np.std(arr))
print("Variance:", np.var(arr))
```

```
Standard Deviation: 14.142135623730951
Variance: 200.0
```

9. Use NumPy functions for mathematical operations

```
import numpy as np

arr = np.array([1, 4, 9, 16])

print("Square Root:", np.sqrt(arr))
print("Log:", np.log(arr))
print("Exponential:", np.exp(arr))
```

```
Square Root: [1.  2.  3.  4.]
Log: [0.          1.38629436  2.19722458  2.77258872]
Exponential: [2.71828183e+00  5.45981500e+01  8.10308393e+03  8.88611052e+06]
```

10. Compare Python lists and NumPy arrays

```
import numpy as np
import time

# Python list
lst = list(range(1000000))
start = time.time()
lst_result = [x * 2 for x in lst]
print("List time:", time.time() - start)

# NumPy array
arr = np.array(lst)
start = time.time()
arr_result = arr * 2
print("NumPy time:", time.time() - start)
```

```
List time: 0.05950522422790527
NumPy time: 0.005092620849609375
```

✓ Bright Students (Advanced)

11. Implement vectorized operations for performance comparison

```
import numpy as np
import time

arr = np.arange(1000000)

# Without vectorization
start = time.time()
result = []
for i in arr:
    result.append(i * 2)
print("Loop time:", time.time() - start)

# With vectorization
start = time.time()
result = arr * 2
print("Vectorized time:", time.time() - start)
```

```
Loop time: 0.20218491554260254
Vectorized time: 0.018832683563232422
```

12. Solve a system of linear equations

```
import numpy as np

A = np.array([[2, 1], [5, 3]])
B = np.array([8, 18])

solution = np.linalg.solve(A, B)
```

```
print("Solution:", solution)
```

```
Solution: [ 6. -4.]
```

13. Perform broadcasting operations

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([10, 20, 30])

result = arr + b

print("Broadcasting Result:\n", result)
```

```
Broadcasting Result:
[[11 22 33]
 [14 25 36]]
```

14. Analyze large datasets using NumPy arrays

```
import numpy as np

data = np.random.randint(1, 100, size=1000)

print("Mean:", np.mean(data))
print("Max:", np.max(data))
print("Min:", np.min(data))
```

```
Mean: 49.593
Max: 99
Min: 1
```

15. Optimize numerical computations using NumPy functions

```
import numpy as np

arr = np.random.rand(1000000)

# Optimized computation
result = np.sqrt(arr) + np.log(arr)

print("Computation Done")
```

```
Computation Done
```

Conclusion

NumPy is a powerful and essential library in Python that plays a vital role in scientific computing and data processing. It provides efficient multi-dimensional array structures and a wide range of mathematical functions, enabling fast and optimized numerical computations. Compared to pure Python, NumPy is significantly faster due to its vectorized operations, optimized C-based implementation, and efficient memory management.

Through this assignment, we explored various mathematical operations such as arithmetic, statistical, trigonometric, and linear algebra functions, along with important concepts like indexing and slicing for effective data access and manipulation. Additionally, NumPy's seamless integration with libraries like Pandas, Matplotlib, and Scikit-learn highlights its importance as the backbone of the Python data science ecosystem.

Overall, NumPy enhances performance, simplifies complex computations, and provides a strong foundation for advanced applications in data analysis, visualization, and machine learning.

